



**Faculty of Mechanical
Engineering and Robotics**

**Department of Robotics and
Mechatronics**



Signal processing and identification in control of mechatronic devices

Topic: Image analysis – chosen topics

Goal: Presentation of chosen image processing and analysis topics, students are introduced to correlation-based techniques of object detection and description of parametric curves by means of Hough transform, introduction to object tracking.

Problems:

1. Image correlation in object detection,
2. Hough transform for line and circle detection,
3. Basics of object tracking algorithms.

Prowadzący: dr inż. Krzysztof Holak

1. Correlation coefficient applied in object detection.

Correlation coefficient may be used in order to find a given pattern in an image. Correlation function takes an image of a pattern and the image in which the pattern has to be found. The function outputs a new matrix in which the values correspond to the degree of similarity between a pattern and an image patch for each of the position of the pattern on the image. Next, the maximum of the correlation coefficient matrix is found, and the position for which the maximum occurs gives the position of the pattern on the image. To compute the similarity map, the following function is used:

```
c = normxcorr2(image_pattern,image);
```

Variable `c` can be displayed as an image or surface using function `surf(c)`. Next, knowing the similarity value as a function of image position, maximum value of the similarity map can be found. In MATLAB, it can be computed using the following code:

```
[max_c, imax] = max(abs(c(:)));  
[ypeak, xpeak] = ind2sub(size(c),imax(1));
```

Remember that `normxcorr2()` function changes the size of the analyzed image, therefore you need to recompute the indices of a maximum:

```
peak_offset = [(xpeak - size(image_pattern,2))  
               (ypeak - size(image_pattern,1))];
```

Problem 1

Write a program that finds a pattern on an image. Modify the program so a set of different patterns can be found. Mark all found patterns on the image. How many instances of the same pattern can be found on the same image?

2. Line detection – Hough transform

Hough transform detects lines in edge images. The method obtains parameter of lines (given for a parametric equation in polar form) detected. Depending on a chosen input parameters, it is possible to find lines of different angles, in a different distances from each other and represented by a different number of edge pixels. There are three functions necessary to perform Hough transform of an image:

- 1) `[H, theta, rho] = hough(im_bw)` – the function takes binary image as an input. The edge image can be obtained using for example, a function **edge()**. The matrix `H` is returned. It contains Hough transform of an image. The other output variables are matrices `rho` and `theta` which store the range of values of `rho` and `theta` parameters. It is possible to use parameters which specify the resolution of `rho` and `theta` (default values are equal to 1).
- 2) `peaks = houghpeaks(H, numpeaks)` – the function computes coordinates of maxima of Hough transform. The variable *numpeaks* specifies how many peaks has to be computed. Additionally the other parameters can be applied: the one which describes threshold above which the value of Hough transform is taken as maximum and another one which gives the size of a neighborhood of a maximum that is excluded from analysis when a maximum has already be found.
- 3) `lines = houghlines(im_bw, theta, rho, peaks)` – function computes the endpoints of line segments which were found by line detection using Hough transform. It takes binary image, range of `theta` and `rho` parameters and positions of found maxima. After computation it returns structure storing found line segments. Other input parameters control processing of found lines – the minimal length of a segment and the distance between two line segments below which they are merged into one segment.

Problems 2.

1. Write a program that uses Hough transform to find lines on a given image. Draw found lines. Check the impact of functions parameters on the performance of the program.
2. Write a program that detects a set of lines of a particular characteristics.

3. Object tracking.

Function **regionprops()** makes it possible to compute object features such as centroid position or bounding rectangle of a shape. Therefore it can be used in a simple object tracking algorithm. In order to track an object on an image sequence, a movie frames have to be load. When avi file is available, the command which reads it is given as:

```
mov = aviread(filename);
```

due to a finite memory capacity, it is recommended to load a sequence frame by frame using loop:

```
for index = 1:endmovie
    mov = aviread(filename, index);
end
```

The variable `mov` is a matrix of structures in which one of the fields contains frames of a sequence in a form of image matrices. In case the sequence is given as a directory with image files, you need to read all the names of graphical files stored in a directory:

```
srcFiles = dir(dirname);
```

Next you can read the images from a directory, one by one.

```
for indexImage = 1:length(srcFiles)

    im_name = strcat(dir, srcFiles(indexImage).name);
    image = imread(im_name);
end
```

Problem 3.

Write a program for object tracking. Plot a graph showing tracked trajectory of an object. Compute velocity of an object numerically. Scale all the results, so all the graphs are in a metric units (millimeters instead of pixels, seconds instead of frame names).

